SAGE

Self-improving Autonomous Generation Engine

Architecture Specification — v1.0 Final

Status: Checkpoint 1 Complete — Implementation Ready

> **Tagline:** *prompt → production*
>
> SAGE converts a natural language goal into a tested, documented software repository
> through coordinated AI agents — running locally by default, with optional cloud fallback.
> It works on greenfield projects, existing codebases, unfinished repos, and bug fixes.
> It is the first open-source agent system with a live orchestrator intelligence feed, two compounding self-improvement loops, and a three-tier RL strategy built in from day one.

Table of Contents

1. Project Vision
2. What Makes SAGE Different
3. Codebase Intelligence Layer
4. Prompt Intelligence Middleware
5. Design Philosophy
6. High-Level Architecture
7. Model Router
8. Task Graph Engine
9. Task Scheduler
10. Agent Orchestrator — LangGraph State Machine
11. Orchestrator Intelligence Feed

12. Event Bus
13. Inter-Agent Communication Protocol
14. Agent Roster
15. Tool Execution Engine + Safety Limits
16. Memory System — 5-Layer Architecture
17. Session Continuity + Handoff Protocol
18. User Rules Layer
19. Debug Loop + Circuit Breaker
20. Human-in-the-Loop Checkpoints
21. System Execution Flow
22. Observability System
23. Benchmark System
24. Technology Stack
25. Repository Structure
26. Development Roadmap
27. Minimum Viable System
28. Self-Improvement Loops
29. Reinforcement Learning Strategy
30. Open Source Strategy
31. Long-Term Vision
32. Quick Start

---

1. Project Vision

SAGE behaves like a **software engineering team composed of AI agents**. Give it an idea, an existing codebase, or an unfinished repo. It understands what exists, plans what's needed,
codes, executes, debugs, and delivers a working repository — without you writing a single line.

```
User Request
     ↓
Codebase Intelligence    ← understands what already exists
     ↓
Prompt Intelligence      ← optimises every LLM call
     ↓
Planning                 ← delta DAG: only what needs doing
```

```
     ↓
Coding
     ↓
Execution
     ↓
Debugging
     ↓
Tested Repository
```

Four Operating Modes

Mode	Input	What SAGE Does
Greenfield	Empty folder + idea	Builds full project from scratch
Feature addition	Existing repo + request	Understands codebase, adds feature in matching style
Bug fix	Existing repo + description	Locates root cause, patches, re-tests
Continuation	Unfinished repo	Reads TODOs and incomplete code, finishes the work

All four modes flow through the same pipeline. The difference is what the Codebase Intelligence Layer hands to the planner.

Example — Greenfield

```
sage run "Build a FastAPI backend with JWT auth and PostgreSQL"
```

Output:

```
backend/
  main.py
  auth.py
  database.py
requirements.txt
Dockerfile
README.md
tests/
```

```
  test_auth.py
  test_api.py
.sage/
  project.json
  codebase_map.json
  conventions.md
.sage-memory.md
memory/system_state.json
```

Example — Continuation

```
sage run "finish what was started" --repo ./my-unfinished-api
```

SAGE scans the repo, finds incomplete functions, missing tests, and TODO comments, then generates a delta Task DAG covering only the remaining work — in the existing code style, using the existing stack.

---

2. What Makes SAGE Different

Every other open-source agent system fails on at least one of these.
SAGE is designed to pass all seven.

Property	Most agent systems	SAGE
Existing repos	Greenfield only	Full codebase understanding
Prompt quality	Raw input → LLM	Every call optimised by middleware
Memory	Starts from zero	5-layer persistent memory
Inter-agent comms	Free-form text	Strict JSON protocol
Failure handling	Loops or crashes	Circuit breaker + fix pattern store
Orchestrator visibility	Blind between events	Live insight feed from every agent
Self-improvement	Static	Two compounding loops + 3-tier RL

3. Codebase Intelligence Layer

Runs at session start when an existing repo is detected.
Produces a context brief that replaces the empty-folder assumption everywhere downstream.

Detection

```
sage run "add rate limiting" --repo ./existing-project
# OR
cd ./existing-project && sage run "add rate limiting"
# SAGE detects .git and triggers codebase intelligence automatically
```

Four Stages

Stage 1 — Structural Scan *(fast, seconds)*

Reads file tree, detects language, framework, entry points, test locations.

```
class CodebaseScanner:
    def scan(self, repo_path: str) -> CodebaseMap:
        return CodebaseMap(
            languages     = self._detect_languages(),
            framework     = self._detect_framework(),    # FastAPI, Django,
            entry_points  = self._find_entry_points(),
            test_locations= self._find_tests(),
            package_files = self._find_deps(),           # requirements.txt
            git_status    = self._read_git_status(),     # untracked, modif
            incomplete    = self._find_todos()           # TODO, FIXME, HAC
        )
```

Stage 2 — Semantic Understanding *(slower, per file)*

Reads code and builds a semantic map. Chunks by logical units using Tree-sitter (AST-based, not line-based) — each function and class becomes a searchable vector.

```
# Each chunk → Qdrant vector
{
```

```
    "file": "auth.py",
    "chunk_type": "function",
    "name": "verify_token",
    "summary": "validates JWT token, raises 401 if expired",
    "dependencies": ["jose", "database.get_user"],
    "complexity": "medium",
    "has_tests": false,
    "embedding": [...]
}
```

This becomes the **project knowledge base** — every agent queries it at any time.

Stage 3 — State Assessment

Reads git history, TODO comments, empty function bodies, missing tests, and broken imports to determine what's complete, partial, or broken.

```
{
    "completion_status": {
        "auth.py":     "complete",
        "database.py": "partial",    # TODO on line 47
        "payments.py": "skeleton",   # functions defined, bodies empty
        "tests/":      "missing"
    },
    "open_threads": [
        "TODO: add rate limiting to /login",
        "FIXME: connection pool leaks on timeout",
        "# placeholder - implement stripe webhook"
    ],
    "last_active_files": ["auth.py", "main.py"],
    "broken_imports":    ["from utils import missing_function"]
}
```

Stage 4 — Context Brief → Planner

Merges all findings into a single structured brief passed to the planner.

```
{
    "mode": "existing_repo",
    "codebase_summary": "FastAPI backend, 60% complete, missing payments +
    "what_exists":       [...],
    "what_is_incomplete":[...],
    "what_is_broken":    [...],
    "suggested_next_tasks": [...],
    "architecture_inferred": "...",   # LLM-generated summary
```

```
    "conventions": {
        "style": "black + isort",
        "patterns": "repository pattern, dependency injection",
        "naming": "snake_case functions, PascalCase models"
    }
}
```

How the Planner Changes

When `mode == "existing_repo"`, the planner generates a **delta Task DAG** — only the work that needs doing, nothing that already works.

```
Greenfield DAG:       setup → scaffold → implement → test → document
Existing repo DAG:    understand → gap_analysis → implement_missing → fix_br
```

The coder reads existing files first and writes new code that matches the style, patterns, and conventions already in the codebase.

The `.sage/` Project Directory

SAGE writes this to every repo it works on. Subsequent sessions read it instead of re-scanning — dramatically faster startup.

```
.sage/
  project.json          ← framework, stack, detected conventions
  codebase_map.json     ← semantic map of all files
  session_history.json  ← every SAGE session on this repo
  conventions.md        ← coding style, patterns, naming rules
  open_threads.md       ← known TODOs, incomplete work, broken things
```

Repository Structure for This Module

```
sage/
  codebase/
    scanner.py           ← structural scan
    semantic_reader.py   ← Tree-sitter AST chunking + LLM summarisation
    state_assessor.py    ← completion detection + broken import finding
    context_builder.py   ← assembles context brief for planner
    conventions.py       ← style + pattern extraction
```

4. Prompt Intelligence Middleware

The most architecturally novel component of SAGE.
Every LLM call — from every agent — passes through this layer without exception.

Pipeline

```
Raw Input (from user or any agent)
        ↓
  Prompt Intelligence Middleware
    ├─ 1. Semantic search → knowledge base
    ├─ 2. Agent-role template injection
    ├─ 3. Recency-weighted successful prompt retrieval
    └─ 4. Keyword fallback
        ↓
  Enhanced Prompt
        ↓
  Model Router
        ↓
  Agent / LLM
```

Knowledge Base (RAG Sources)

- Anthropic Claude prompting documentation
- OpenAI prompting guide
- Google Gemini documentation
- Prompt engineering research papers (academic + DeepMind)
- Internal agent performance logs (ranked by output quality score)
- Agent-role-specific prompt templates
- Codebase conventions (from `.sage/conventions.md` for existing repos)

Retrieval Strategy

Priority	Method
1	Semantic similarity against knowledge base

Priority	Method
2	Recency-weighted: recently successful prompts rank higher
3	Agent-role template injection (planner ≠ coder ≠ debugger)
4	Keyword fallback

Universal Prompt Template

Every agent receives its prompt in this exact format. No exceptions.

```
SYSTEM:
You are the SAGE {agent_role} agent.
Your output must be valid JSON matching the {output_schema} schema.
Never explain your reasoning in prose. Return structured output only.

TASK:
{task_description}

PROJECT CONTEXT:
{project_memory_summary}

RECENT SESSION STATE:
{system_state_summary}

CODEBASE CONTEXT:
{codebase_brief_if_existing_repo}

ORCHESTRATOR NOTES:
{orchestrator_injected_context}

TOOL PERMISSIONS:
{allowed_tools_for_this_task}

KNOWN PATTERNS:
{relevant_fix_patterns_if_applicable}

EPISTEMIC RULES:
- Tag inferred decisions with [INFERRED]
- Tag untested outputs with [UNVERIFIED]
- Tag ambiguous inputs with [UNCLEAR] and surface them in your output
```

```
OUTPUT FORMAT:
{output_schema_example}
```

Agent-Specific Templates

```
prompt_engine/templates/
  planner.md            ← structured task decomposition, delta DAG mode
  architect.md          ← folder structure + tech decisions
  coder.md              ← PatchRequest output, matches existing conventions
  debugger.md           ← error analysis, suspected_cause required
  reviewer.md           ← security + logic checks
  memory_optimizer.md   ← compression rules, no hallucination
```

---

5. Design Philosophy

Priority	Principle
1	**Deterministic workflows** — no chaotic autonomous loops
2	**Structured communication** — strict JSON between all agents
3	**Observability** — every decision, action, and failure is logged
4	**Reproducibility** — same prompt produces same result
5	**Tool-first agents** — agents request actions, never execute directly
6	**Self-improving memory** — system learns measurably from its failures
7	**Safety by default** — all execution is bounded and sandboxed
8	**Context-aware** — works on existing codebases, not just blank slates

Architecture style:

```
Goal-Driven Planning
+ Task Graph Execution
+ Event-Driven Recovery
+ Persistent Session Memory
```

```
+ Live Orchestrator Supervision
+ Compounding Self-Improvement
```

---

6. High-Level Architecture

```
sage run "your request" [--repo /path/to/existing]
          ↓
┌──────────────────────────────────────────────────────┐
│  IF existing repo detected                           │
│  Codebase Intelligence Layer                         │
│    ├─ structural scan (seconds)                      │
│    ├─ semantic understanding → Qdrant                │
│    ├─ state assessment (complete/partial/broken)     │
│    └─ context brief → planner                        │
│  IF greenfield: skip                                 │
└──────────────────────────────────────────────────────┘
          ↓
sage-context.sh ──────────────────── load system_state + git context
          ↓
Prompt Intelligence Middleware ──── wraps every LLM call
          ↓
Model Router ──────────────────────── correct model per task type
          ↓
Planner Agent ─────────────────────── produces delta Task DAG
          ↓
Human Checkpoint ──────────────────── approve / edit / reject
          ↓
Task Scheduler ────────────────────── MAX_PARALLEL=3, dep resolution
          ↓
Agent Orchestrator (LangGraph) ──── state machine
          ↓              ↕                    ↕
  Memory Manager    Event Bus    Orchestrator Intelligence Feed
          ↓
Parallel Agent Workers ──────────── Coder / Architect / Reviewer
  (each emits AgentInsight packets continuously)
          ↓
Tool Execution Engine ──────────── sandboxed + safety limits
          ↓
Fix Pattern Store check ─────────── skip debug if pattern known
          ↓  (no match)
Debug Loop ────────────────────────── patch → retry
```

```
          ↓  (max retries)
Circuit Breaker ──────────────────── escalate / skip / halt
          ↓
SESSION_END ────────────────────────── update memory + .sage/ + git hook
          ↓
Output Repository + Benchmark Log
```

---

7. Model Router

SAGE never lets agents randomly select models. Every task type has a designated primary model with an explicit fallback chain and trigger conditions.

Routing Table

Agent	Primary (Local)	Fallback (Cloud)	Reason
Planner	Llama 3	Claude	Reasoning-heavy, instruction following
Architect	Llama 3	Claude	Design decisions, structured output
Coder	DeepSeek-Coder	GPT-4o	Code generation specialist
Debugger	Codestral	GPT-4o	Code repair specialist
Reviewer	DeepSeek-Coder	Claude	Code analysis + security
Test Engineer	DeepSeek-Coder	GPT-4o	Test generation
Memory Optimizer	Llama 3	Claude	Compression + summarisation
Documentation	Llama 3	Claude	Natural language generation

Routing Config (`config/models.yaml`)

```yaml
routing:
  planner:
    primary: "llama3:8b"
    fallback: "claude-sonnet-4-5"
    fallback_triggers:
      - task_complexity_score > 0.8
      - primary_failure_count >= 2
  coder:
    primary: "deepseek-coder:6.7b"
    fallback: "gpt-4o"
    fallback_triggers:
      - language not in ["python", "javascript", "typescript"]
      - primary_failure_count >= 2
  debugger:
    primary: "codestral:latest"
    fallback: "gpt-4o"
    fallback_triggers:
      - primary_failure_count >= 2
```

---

8. Task Graph Engine

All work in SAGE is modelled as a DAG. No agent acts outside the graph.
This single rule prevents 90% of multi-agent chaos.

Example DAGs

```
Greenfield:                Existing repo (continuation):
setup_project              understand_codebase
      ↓                          ↓
create_backend             gap_analysis
      ↓                          ↓
connect_database           implement_missing
      ↓                          ↓
implement_auth             fix_broken_imports
      ↓                          ↓
add_tests                  add_missing_tests
```

TaskNode Schema

```
{
  "id": "string",
  "description": "string",
  "dependencies": ["task_id"],
  "assigned_agent": "coder | architect | reviewer",
  "status": "pending | running | blocked | failed | completed",
  "retry_count": 0,
  "model_used": "deepseek-coder:6.7b",
  "mode": "greenfield | existing_repo",
  "epistemic_flags": ["[INFERRED]", "[UNCLEAR]", "[UNVERIFIED]"]
}
```

Epistemic Flags

Flag	Meaning	Orchestrator Response
`[INFERRED]`	Agent assumed something not specified	Log + annotate downstream
`[UNVERIFIED]`	Output produced but not tested	Force test before mark complete
`[UNCLEAR]`	Input ambiguous	Surface to human checkpoint

---

9. Task Scheduler

Controls execution order and parallelism limits.

Algorithm

```
class TaskScheduler:
    MAX_PARALLEL = 3
    MAX_QUEUE_SIZE = 10
```

```python
    def get_ready_tasks(self, dag: TaskGraph) -> list[TaskNode]:
        return [
            task for task in dag.nodes
            if task.status == "pending"
            and all(dag.get(dep).status == "completed"
                    for dep in task.dependencies)
        ]

    def schedule_next(self, dag, running) -> list[TaskNode]:
        slots = self.MAX_PARALLEL - len(running)
        return self.get_ready_tasks(dag)[:slots]
```

Parallelism Rules

Scenario	Behaviour
Independent tasks	Run in parallel, up to MAX_PARALLEL=3
Dependent tasks	Wait for upstream completion
Failed dependency	Downstream tasks → BLOCKED (not FAILED)
All tasks blocked	Emit `PIPELINE_BLOCKED` → human checkpoint

---

10. Agent Orchestrator — LangGraph State Machine

The central nervous system. Coordinates all agents, manages state, routes events, and acts as an active supervisor through the Orchestrator Intelligence Feed.

State Object

```python
class SAGEState(TypedDict):
    user_prompt: str
    enhanced_prompt: str
    repo_mode: str                  # "greenfield" | "existing_repo"
    codebase_brief: dict            # from Codebase Intelligence Layer
    task_dag: dict
    current_task: TaskNode
```

```
    agent_output: dict
    execution_result: dict
    debug_attempts: int
    session_memory: dict
    insight_buffer: dict            # live AgentInsight packets
    events: list[Event]
```

Graph Definition

```
workflow = StateGraph(SAGEState)

workflow.add_node("detect_mode",         detect_greenfield_or_existing)
workflow.add_node("codebase_intel",      run_codebase_intelligence)
workflow.add_node("load_memory",         load_session_memory)
workflow.add_node("prompt_middleware",   run_prompt_intelligence)
workflow.add_node("route_model",         run_model_router)
workflow.add_node("planner",             run_planner_agent)
workflow.add_node("human_checkpoint",    run_human_checkpoint)
workflow.add_node("scheduler",           run_task_scheduler)
workflow.add_node("execute_agent",       run_assigned_agent)
workflow.add_node("tool_executor",       run_tool_execution)
workflow.add_node("check_fix_patterns",  check_fix_pattern_store)
workflow.add_node("debug_agent",         run_debug_agent)
workflow.add_node("circuit_breaker",     run_circuit_breaker)
workflow.add_node("save_memory",         save_session_state)

workflow.set_entry_point("detect_mode")

workflow.add_conditional_edges(
    "detect_mode",
    decide_mode,
    {
        "existing_repo": "codebase_intel",
        "greenfield":    "load_memory",
    }
)

workflow.add_edge("codebase_intel",    "load_memory")
workflow.add_edge("load_memory",       "prompt_middleware")
workflow.add_edge("prompt_middleware", "route_model")
workflow.add_edge("route_model",       "planner")
workflow.add_edge("planner",           "human_checkpoint")
workflow.add_edge("human_checkpoint",  "scheduler")
workflow.add_edge("scheduler",         "execute_agent")
```

```
workflow.add_edge("execute_agent",     "tool_executor")

workflow.add_conditional_edges(
    "tool_executor",
    decide_after_execution,
    {
        "success":      "scheduler",
        "test_failed":  "check_fix_patterns",
        "build_error":  "check_fix_patterns",
        "all_done":     "save_memory",
    }
)

workflow.add_conditional_edges(
    "check_fix_patterns",
    decide_after_pattern_check,
    {
        "pattern_found": "tool_executor",
        "no_pattern":    "debug_agent",
    }
)

workflow.add_conditional_edges(
    "debug_agent",
    decide_after_debug,
    {
        "patched":      "tool_executor",
        "max_retries":  "circuit_breaker",
    }
)

workflow.add_conditional_edges(
    "circuit_breaker",
    decide_after_circuit_break,
    {
        "skip":     "scheduler",
        "escalate": "human_checkpoint",
        "halt":     "save_memory",
    }
)
```

---

11. Orchestrator Intelligence Feed

The feature that turns the orchestrator from a scheduler into an active supervisor.

Currently, the orchestrator only receives terminal events — `TASK_COMPLETED`, `TEST_FAILED`.
It is blind to everything happening *during* execution: what the coder flagged as risky, why the architect made a structural choice, what the debugger suspects before it patches.

The Orchestrator Intelligence Feed closes that gap. Every agent emits structured `AgentInsight` packets continuously. The orchestrator reads them in real time and can pre-empt failures, reassign tasks, or inject context into downstream agents *before* problems surface.

What This Enables

- Coder flags a security vulnerability mid-task → orchestrator injects a security note into the reviewer's prompt before review starts
- Planner emits three `[UNCLEAR]` flags → orchestrator escalates to human *before* any code is written, not after three failed debug cycles
- Debugger suspects a dependency conflict → orchestrator annotates the executor to check the environment before re-running tests
- Architect marks a structural decision `[INFERRED]` → orchestrator logs it as a known assumption all future agents will see

Orchestrator Intelligence Feed Implementation

```
class OrchestratorIntelligenceFeed:
    def __init__(self):
        self.insight_buffer: dict[str, list[AgentInsight]] = {}
        self.pending_context: dict[str, list[str]] = {}
        self.intervention_thresholds = {
            "uncertainty_count": 3,     # 3+ [UNCLEAR] → escalate before co
            "high_severity_count": 1,   # any high → intervene immediately
            "risk_accumulation": 0.7    # composite risk score → human chec
        }

    async def ingest(self, insight: AgentInsight):
        self.insight_buffer[insight.task_id].append(insight)
        await self._evaluate(insight)
```

```
    async def _evaluate(self, insight: AgentInsight):
        if insight.severity == "high" or insight.requires_orchestrator_acti
            await self._intervene(insight)
        elif insight.severity == "medium":
            await self._annotate_downstream(insight)
        else:
            await self._observe(insight)   # log only

    async def _annotate_downstream(self, insight: AgentInsight):
        self.pending_context[insight.task_id].append(
            f"ORCHESTRATOR_NOTE [{insight.agent}]: {insight.content}"
        )

    async def _intervene(self, insight: AgentInsight):
        # reassign model, add checkpoint, or pause task branch
        await self.event_bus.emit(Event(
            type="ORCHESTRATOR_INTERVENTION",
            task_id=insight.task_id,
            payload={"reason": insight.content, "severity": insight.severit
        ))
```

Orchestrator Actions by Severity

Severity	Action
`low`	Log to session journal + annotate downstream agent context
`medium`	Inject into next agent's prompt as `ORCHESTRATOR_NOTE`
`high`	Reassign model / add mid-pipeline checkpoint / adjust task
`requires_action: true`	Pause task branch → human escalation immediately

---

12. Event Bus

All orchestration is event-driven. No agent polls. Everything reacts.

Implementation

```python
# orchestrator/event_bus.py
import asyncio
from dataclasses import dataclass

@dataclass
class Event:
    type: str
    task_id: str
    payload: dict
    timestamp: str

class EventBus:
    def __init__(self):
        self._handlers: dict[str, list] = {}
        self._queue = asyncio.Queue()

    def subscribe(self, event_type: str, handler):
        self._handlers.setdefault(event_type, []).append(handler)

    async def emit(self, event: Event):
        await self._queue.put(event)

    async def process(self):
        while True:
            event = await self._queue.get()
            for handler in self._handlers.get(event.type, []):
                await handler(event)
```

Upgrade path: async queue → Redis pub/sub when SAGE runs distributed.

Event Registry

Event	Emitted By	Handled By	Action
`TASK_COMPLETED`	Executor	Scheduler	Advance DAG
`TASK_FAILED`	Circuit Breaker	Orchestrator	Escalate / skip

Event	Emitted By	Handled By	Action
`TEST_FAILED`	Executor	Debug Agent	Start debug loop
`BUILD_ERROR`	Executor	Debug Agent	Start debug loop
`PATCH_APPLIED`	Tool Engine	Executor	Re-run tests
`PIPELINE_BLOCKED`	Scheduler	Orchestrator	Human checkpoint
`MEMORY_CHECKPOINT`	Orchestrator	Memory Manager	Write session log
`SESSION_END`	Orchestrator	Memory Manager	Update system_state.json
`PATTERN_LEARNED`	Debug Agent	Fix Pattern Store	Write new pattern
`CIRCUIT_BREAKER_FIRED`	Debug Loop	Orchestrator	Escalate / skip / halt
`ORCHESTRATOR_INTERVENTION`	Intel Feed	Orchestrator	Reassign / checkpoint
`AGENT_INSIGHT`	Any Agent	Intel Feed	Ingest + evaluate

---

13. Inter-Agent Communication Protocol

All agents communicate through strict JSON schemas. No free-form text between agents.
This is the single rule that prevents hallucinated formats from breaking the pipeline.

TaskResult

```
{
  "task_id": "string",
  "status": "completed | failed",
  "summary": "string",
  "artifacts": ["file paths"],
  "epistemic_flags": ["[INFERRED]", "[UNVERIFIED]"],
  "model_used": "deepseek-coder:6.7b",
  "tokens_used": 1240,
  "logs": "optional stdout/stderr"
}
```

PatchRequest

```
{
  "file": "path/to/file",
  "operation": "edit | create | delete",
  "patch": "unified diff or full content",
  "reason": "why this change is necessary",
  "matches_existing_conventions": true,
  "epistemic_flags": ["[INFERRED]"]
}
```

ErrorReport

```
{
  "task_id": "string",
  "error_type": "runtime | test | dependency | logic",
  "logs": "full stack trace",
  "suspected_cause": "agent analysis",
  "suggested_fix": "optional patch direction",
  "pattern_match": "error_signature if known fix exists"
}
```

AgentInsight ← New

Emitted continuously by every agent during execution.
Received and evaluated by the Orchestrator Intelligence Feed.

```
{
  "agent": "coder | planner | architect | debugger | reviewer",
```

```
  "task_id": "string",
  "insight_type": "uncertainty | risk | decision | observation",
  "content": "JWT secret key is hardcoded in main.py — flagging before comm
  "severity": "low | medium | high",
  "epistemic_flag": "[INFERRED] | [UNCLEAR] | [UNVERIFIED]",
  "timestamp": "2026-03-16T14:22:00Z",
  "requires_orchestrator_action": false
}
```

---

14. Agent Roster

MVP Agents

Agent	Input	Output Schema	Primary Model
Planner	User prompt + codebase brief	Task DAG + project spec	Llama 3
Architect	Task DAG	Folder structure + tech decisions	Llama 3
Coder	TaskNode + conventions	PatchRequest	DeepSeek-Coder
Executor	PatchRequest	ExecutionResult	— (tool call)
Debugger	ErrorReport	PatchRequest	Codestral

Phase 2 Agents

Agent	Role
Reviewer	Code quality + security (ruff, bandit, mypy)
Test Engineer	Generate pytest + integration tests
Memory Optimizer	Weekly compression + pattern promotion

Agent	Role
Documentation	README + API docs

---

15. Tool Execution Engine + Safety Limits

All agent actions route through this engine. Nothing touches the host system directly.

Safety Config (`config/pipeline.yaml`)

```yaml
safety:
  max_command_time_seconds: 30
  max_file_write_bytes: 5242880       # 5MB
  max_patch_lines: 200
  max_pip_install_packages: 10
  blocked_commands:
    - "rm -rf /"
    - "sudo"
    - "curl | bash"
    - "wget | sh"
  max_concurrent_processes: 3
  docker_memory_limit: "512m"
  docker_cpu_limit: "1.0"
```

Execution Handler

```python
class ToolExecutionEngine:
    def execute(self, req: PatchRequest) -> ExecutionResult:
        self._safety_check(req)

        if req.operation in ["edit", "create", "delete"]:
            return self._filesystem_handler(req)
        elif req.operation == "run_command":
            return self._terminal_handler(req)

    def _terminal_handler(self, req):
        result = subprocess.run(
            req.command,
```

```
            timeout=self.safety.max_command_time_seconds,
            capture_output=True,
            shell=False,            # never shell=True
        )
        return ExecutionResult(stdout=result.stdout, returncode=result.retu
```

Supported Operations

Category	Operations	Isolation
Filesystem	read, write, patch, delete	subprocess
Terminal	install, run, build, test	Docker
Git	init, commit, branch, diff	subprocess
Runtime	pytest, npm test, docker build	Docker
Memory	read/write state, append logs	direct

---

16. Memory System — 5-Layer Architecture

```
memory/
  system_state.json         ← Layer 1: Active brain (read every session)
  sessions/
    2026-03-16.log          ← Layer 2: Append-only execution journals
  projects/
    project-name.md         ← Layer 3: Scoped project context
  fixes/
    error_patterns.json     ← Layer 4: Self-learned fix patterns
  weekly/
    2026-W11.md             ← Layer 5: Compressed weekly digests
.sage-memory.md             ← Per-repo: auto-updated by git hook
.sage/                      ← Per-repo: codebase intelligence cache
```

Layer 1 — System State

Read by the orchestrator before every session. Written at every `SESSION_END`.

```
{
  "last_updated": "2026-03-16T14:30:00Z",
  "active_project": "fastapi-auth-backend",
  "repo_mode": "existing_repo",
  "last_completed_task": "connect_database",
  "next_unblocked_task": "implement_auth",
  "open_blockers": ["PostgreSQL connection string not set"],
  "session_count": 4,
  "overall_success_rate": 0.78,
  "fix_pattern_hit_rate": 0.34
}
```

Layer 2 — Session Journals

Append-only. Checkpoint every 5 completed tasks.

```
[2026-03-16 14:22] CHECKPOINT
  Completed: setup_project, create_backend
  In progress: connect_database
  Last file: backend/database.py
  Last command: pip install sqlalchemy
  Model: deepseek-coder:6.7b
  Insights logged: 3 (2 low, 1 medium — orchestrator annotated executor)
  Flags: [UNVERIFIED] database connection not tested yet
```

Layer 3 — Project Memory

Architecture decisions, file structure, known issues — scoped per project.

Layer 4 — Self-Learned Fix Patterns

```
{
  "error_signature": "ModuleNotFoundError: No module named 'sqlalchemy'",
  "fix": "pip install sqlalchemy psycopg2-binary",
  "success_rate": 0.95,
  "times_applied": 7,
  "last_used": "2026-03-15",
  "source": "debug_agent"
}
```

Before the debug loop runs, the fix pattern store is checked first.
Match found → apply immediately, skip debug agent entirely.

Layer 5 — Weekly Digest

Memory Optimizer Agent runs weekly. Compresses logs, removes stale entries (>14 days untouched), updates system_state.json, writes digest.

Git Hook

`.git/hooks/post-commit`

```
#!/bin/bash
echo "$(date '+%Y-%m-%d %H:%M') | $(git log -1 --oneline)" >> .sage-memory.
```

Context Injection Script

`sage-context.sh`

```
#!/bin/bash
CONTEXT=$(cat <<EOF
$(cat memory/system_state.json 2>/dev/null || echo "No prior state")
$(cat .sage/project.json 2>/dev/null || echo "")
Recent commits: $(git log --oneline -5 2>/dev/null)
Modified files: $(git diff --name-only HEAD 2>/dev/null)
Current branch: $(git branch --show-current 2>/dev/null)
EOF
)
```

Memory Retrieval Strategy

Query Type	Strategy
Recent context	Recency-weighted (last N sessions)
Similar errors	Semantic vector search (Qdrant)
Architecture decisions	Keyword lookup in project memory

Query Type	Strategy
Fix patterns	Exact signature match → semantic fallback
Codebase understanding	Qdrant vector search over chunked code

Stack: Qdrant + SQLite + flat files

---

17. Session Continuity + Handoff Protocol

SAGE maintains context not just across planned sessions but across unexpected interruptions — model overload, timeout, crash, or manual abort. When a session ends for any reason, a handoff snapshot is written immediately so a new session resumes at the exact point of interruption, not just the last completed task.

The Problem This Solves

system_state.json tracks what tasks are done. That is not enough.

If a coder agent is mid-reasoning on task 6 of 12 and the model is overloaded, a new session reading only system_state.json will restart task 6 from scratch — losing the partial analysis, the in-progress patch, and the context the agent had built up.

The handoff protocol captures the full mid-task state.

Handoff Snapshot

Written to `memory/handoff.json` on any session interruption. Deleted on clean SESSION_END.

```
{
  "interrupted_at": "2026-03-16T14:22:31Z",
  "interrupt_reason": "model_overload | timeout | crash | manual_abort",
  "active_task": {
    "id": "implement_auth",
```

```
    "status": "in_progress",
    "agent": "coder",
    "model": "deepseek-coder:6.7b"
  },
  "partial_output": {
    "files_written": ["backend/auth.py"],
    "files_pending": ["backend/tokens.py"],
    "last_patch_applied": "unified diff...",
    "mid_reasoning": "was analysing JWT expiry edge case on line 47"
  },
  "insight_buffer_snapshot": [...],
  "orchestrator_notes_pending": [...],
  "retry_count": 1,
  "session_context": "full system_state + recent journal entries"
}
```

Parallel Session Management

When a model is overloaded mid-session, SAGE:

1. Writes handoff.json immediately (before the session dies)
2. Emits `SESSION_INTERRUPTED` event to the event bus
3. New session detects handoff.json on startup
4. Loads full handoff context — not just system_state
5. Resumes from `partial_output.mid_reasoning`, not task start
6. Routes to fallback model if primary still overloaded

```
class SessionManager:
    def on_startup(self):
        if Path("memory/handoff.json").exists():
            return self._resume_interrupted_session()
        return self._start_fresh_session()

    def _resume_interrupted_session(self):
        handoff = json.load(open("memory/handoff.json"))
        # restore full context — task state, partial output,
        # insight buffer, orchestrator notes — everything
        return SAGEState(
            current_task    = handoff["active_task"],
            partial_output  = handoff["partial_output"],
            insight_buffer  = handoff["insight_buffer_snapshot"],
            resume_mode     = True
        )
```

```
    def on_interrupt(self, state: SAGEState, reason: str):
        # called by signal handler, timeout watchdog, or model error
        self._write_handoff(state, reason)
        self.event_bus.emit(Event(type="SESSION_INTERRUPTED", ...))
```

Model Overload Routing

When the primary model returns an overload error during a session:

```
Primary model overloaded
      ↓
SessionManager.on_interrupt() called
      ↓
handoff.json written with full mid-task state
      ↓
Model Router checks fallback availability
      ↓
New session spins up with fallback model
      ↓
Resumes from handoff — same task, same context, different model
```

This means a DeepSeek-Coder overload transparently continues on GPT-4o with zero context loss. The user sees no interruption.

Event Addition

Event	Trigger	Handler
`SESSION_INTERRUPTED`	Any unclean exit	SessionManager writes handoff
`SESSION_RESUMED`	Handoff detected on startup	Load full context + notify user
`HANDOFF_CLEARED`	Clean SESSION_END	Delete handoff.json

Repository Addition

```
sage/
  orchestrator/
    session_manager.py    ← handoff write/read, resume logic
  memory/
    handoff.json          ← exists only during interruption
```

---

18. User Rules Layer

SAGE supports user-defined rules that constrain agent behaviour across all sessions — equivalent to Cursor Rules or Claude's system prompt customisation. Rules are loaded before every agent call and injected into the prompt template as a `USER_RULES` block.

What User Rules Control

- Coding style preferences beyond what auto-detection finds
- Libraries or patterns to always use or always avoid
- Security requirements specific to the project
- Output format preferences
- Workflow constraints (e.g. "always write tests before code")
- Agent personality and verbosity
- Domain-specific knowledge the agents should assume

Rule Files — Three Levels

Rules are scoped and merged in priority order.

```
~/.sage/rules.md              ← Global: applies to all projects
.sage/rules.md                ← Project: applies to this repo only
.sage/rules.{agent}.md        ← Agent-specific: only for named agent
```

Lower scope overrides higher scope on conflict.
Agent-specific rules override project rules for that agent only.

Rule File Format

Plain markdown. No special syntax required. Agents read it as natural language.

Example: `~/.sage/rules.md` (global)

```
## Coding Style
- Always use type hints in Python
- Prefer dataclasses over dicts for structured data
- Never use mutable default arguments

## Libraries
- Use httpx not requests for HTTP calls
- Use pydantic v2 not v1
- Never use print() — use structlog

## Testing
- Write tests before implementation (TDD)
- Minimum 80% coverage on new files
- Always include edge case tests

## Security
- Never hardcode secrets — always use environment variables
- Always validate and sanitise user input
- Use parameterised queries, never string concatenation for SQL

## Output
- Keep functions under 30 lines
- One responsibility per function
- Add docstrings to all public functions
```

Example: `.sage/rules.md` (project-specific)

```
## This Project
- Stack: FastAPI + PostgreSQL + Redis
- Auth: JWT only, no sessions
- API versioning: always prefix routes with /api/v1
- Error responses: always use the ErrorResponse schema in schemas.py
- Never modify the User model directly — use UserRepository
```

Example: `.sage/rules.coder.md` (agent-specific)

```
## Coder Agent Rules
- Always check if a similar function already exists before creating a new o
- Match the import style already used in the file being edited
- When editing an existing file, preserve all existing comments
```

How Rules Are Injected

Rules are loaded by the Prompt Intelligence Middleware and inserted into every agent prompt as a `USER_RULES` block — after project context, before tool permissions.

```
SYSTEM: You are the SAGE {agent_role} agent...

TASK: {task_description}

PROJECT CONTEXT: {project_memory_summary}

USER RULES:               ← injected here, every call
{merged_rules_for_this_agent}

ORCHESTRATOR NOTES: {orchestrator_injected_context}

TOOL PERMISSIONS: {allowed_tools}

KNOWN PATTERNS: {fix_patterns}

OUTPUT FORMAT: {schema_example}
```

Rule Conflict Resolution

```python
class UserRulesManager:
    def load_for_agent(self, agent_role: str) -> str:
        global_rules  = self._read("~/.sage/rules.md")
        project_rules = self._read(".sage/rules.md")
        agent_rules   = self._read(f".sage/rules.{agent_role}.md")

        # merge: agent > project > global on conflict
        return self._merge(global_rules, project_rules, agent_rules)
```

Rule Validation

On `sage run`, rules are validated before execution:

- Checks for contradictions (e.g. “always use requests” and “never use requests”)
- Flags rules that conflict with safety limits (e.g. “ignore security warnings”)
- Warns on overly broad rules that may reduce agent effectiveness

CLI Commands

```
# View active rules for current project
sage rules

# View rules for specific agent
sage rules --agent coder

# Add a rule interactively
sage rules add "always use async functions for IO operations"

# Check for conflicts
sage rules validate
```

Repository Addition

```
sage/
  prompt_engine/
    rules_manager.py      ← load, merge, validate user rules
~/.sage/
  rules.md                ← global user rules (created on first run)
.sage/
  rules.md                ← project rules (user-created)
  rules.coder.md          ← agent-specific (optional)
  rules.debugger.md
  rules.planner.md
```

19. Debug Loop + Circuit Breaker

Debug Loop

```
Execute code
      ↓
Observe result
      ↓
Check fix pattern store ── match → apply fix, emit PATTERN_LEARNED, re-exec
      ↓ (no match)
Debug Agent → ErrorReport analysis (suspected_cause required)
      ↓
PatchRequest emitted
```

```
      ↓
Tool Execution Engine (safety checked)
      ↓
Re-execute
      ↓
Write to session journal
      ↓
retry_count++  →  if >= 5: Circuit Breaker
```

Circuit Breaker

```
max_retries_per_task = 5

Decision tree on breach:
  IF fix_pattern_exists AND not yet tried → apply, reset count
  ELIF human_checkpoint_enabled           → escalate to human
  ELIF mode == "auto"                     → skip + mark BLOCKED + continue
  ELSE                                    → halt pipeline
```

All circuit breaker activations are written to the session journal with full error sequence and attempted fixes. This becomes training data for the fix pattern store and the Tier 2 RL trajectory dataset.

---

20. Human-in-the-Loop Checkpoints

Checkpoint	Trigger	Human Options
1 — Post-scan	After codebase intel (existing repos)	Confirm understanding / correct
2 — Post-planning	After Planner produces DAG	Approve / Edit / Reject
3 — Post-failure	Circuit breaker fires	Guide / Skip / Abort
4 — Pre-deploy	Before destructive operations	Confirm / Cancel
5 — Insight escalation	Orchestrator Intel Feed raises `requires_action`	Review / Override

Run modes:

- `sage run --research` → all checkpoints mandatory
- `sage run --auto` → checkpoint only on circuit breaker and insight escalation
- `sage run --silent` → no checkpoints, skip failed tasks automatically

---

21. System Execution Flow

```
sage run "your request" [--repo ./existing]
          ↓
detect_mode
  ├─ existing repo → Codebase Intelligence Layer
  │     ├─ structural scan
  │     ├─ semantic understanding → Qdrant
  │     ├─ state assessment
  │     └─ context brief
  └─ greenfield → skip
          ↓
sage-context.sh → load system_state + git + .sage/
          ↓
Prompt Intelligence Middleware → RAG + template injection
          ↓
Model Router → assign model per agent role
          ↓
Planner → delta Task DAG (only what needs doing)
          ↓
Human Checkpoint 2
          ↓
Task Scheduler → resolve deps, queue MAX_PARALLEL=3
          ↓
For each task:
  ├─ Memory Manager → load context
  ├─ Prompt Intelligence Middleware → enhance
  ├─ Model Router → assign model
  ├─ Assigned Agent → emit PatchRequest + AgentInsight stream
  │     └─ Orchestrator Intelligence Feed ingests insights in real time
  │           ├─ low: log + annotate downstream
  │           ├─ medium: inject ORCHESTRATOR_NOTE into next prompt
  │           └─ high: intervene / checkpoint
  ├─ Tool Execution Engine → apply patch (safety checked)
  ├─ Execute tests
```

```
  ├─ success → TASK_COMPLETED → scheduler advances
  └─ failure:
        ├─ check fix pattern store → match → apply → re-execute
        └─ no match → Debug Agent → patch → retry
              └─ max retries → Circuit Breaker → escalate/skip/halt
          ↓
  MEMORY_CHECKPOINT every 5 tasks
          ↓
SESSION_END
  ├─ update system_state.json
  ├─ append session journal
  ├─ update .sage/ (if existing repo)
  ├─ git post-commit hook
  └─ emit metrics to benchmark log
          ↓
Output Repository
```

---

22. Observability System

What Gets Logged

- Prompt before AND after middleware transformation (quality delta measurement)
- Model routing decisions with reason
- Task status transitions with timestamps
- All tool calls and outputs
- Full `AgentInsight` stream per task (what every agent flagged, when)
- Orchestrator interventions (what triggered them, what action was taken)
- Debug loop iterations with patch diffs
- Circuit breaker activations with full error context
- Fix pattern store hits and misses
- Memory reads and writes
- Token usage per agent call

Per-Run Metrics

```
{
  "run_id": "uuid",
  "repo_mode": "existing_repo | greenfield",
```

```
  "prompt": "user input",
  "tasks_total": 8,
  "tasks_completed": 7,
  "tasks_failed": 1,
  "debug_loop_iterations": 3,
  "circuit_breaker_activations": 0,
  "fix_pattern_hits": 2,
  "orchestrator_interventions": 1,
  "agent_insights_emitted": 14,
  "total_time_seconds": 142,
  "models_used": {
    "deepseek-coder:6.7b": 6,
    "llama3:8b": 2
  },
  "prompt_quality_delta": 0.23,
  "local_vs_cloud_ratio": 0.9
}
```

Future: SAGE Dashboard

- Live task graph (nodes: pending / running / failed / completed)
- Real-time agent insight feed (what every agent is thinking)
- Orchestrator intervention log
- Debug loop trace with patch diffs
- Prompt before/after comparison with quality score
- Fix pattern learning curve over time
- Codebase understanding visualiser (semantic map of ingested repo)

---

23. Benchmark System

Standard Task Suite

Task	Mode	Complexity	Success Criteria
Generate REST API	Greenfield	Low	pytest passes, server starts
Build CRUD web app	Greenfield	Medium	all endpoints return 200

Task	Mode	Complexity	Success Criteria
Add feature to existing repo	Existing	Medium	no regressions, feature works
Fix GitHub issue	Existing	Medium	tests pass after patch
Finish unfinished repo	Continuation	High	all TODOs resolved, tests pass
Build full-stack app	Greenfield	High	frontend + backend + DB running

Metrics

Metric	Definition	Research Value
Build success rate	% of runs producing working code	Primary
Test pass rate	% of generated tests that pass	Code quality
Debug loop iterations	Average iterations to fix failures	Efficiency
Fix pattern hit rate	% of errors resolved from memory	Self-improvement
Prompt quality delta	Output quality before vs after middleware	**Novel — publishable**
Orchestrator intervention rate	How often Intel Feed pre-empts failures	Supervision quality
Codebase understanding accuracy	Correct identification of existing patterns	Ingestion quality
Local vs cloud ratio	% of calls resolved locally	Privacy + cost

---

24. Technology Stack

Local Runtime

Tool	Role
Ollama	Local LLM serving
DeepSeek-Coder 6.7B	Primary coding model
Codestral	Debugger model
Llama 3 8B	Planning + reasoning
Tree-sitter	AST-based code parsing for codebase ingestion

Cloud Fallback

Model	Use Case
Claude	Complex planning, review
GPT-4o	Hard debugging, unfamiliar languages
Gemini	Documentation generation

Orchestration

Tool	Role
LangGraph	State machine orchestration
obra/superpowers	Workflow skills (TDD, debug methodology)
thedotmack/claude-mem	Session memory (adapted for multi-agent)

Memory & Storage

Tool	Role
Qdrant	Vector search (prompts + code chunks + error patterns)

Tool	Role
SQLite	Task graph state + metadata
Flat files	system_state.json, session logs, fix patterns

Execution & Safety

Tool	Role
Python subprocess	Fast local execution
Docker	Isolated execution
pytest	Test runner
git	Version control + commit hooks

---

25. Repository Structure

```
sage/
  agents/
    planner.py
    architect.py
    coder.py
    debugger.py
    reviewer.py
    test_engineer.py
    memory_optimizer.py

  orchestrator/
    workflow.py              ← LangGraph state machine
    task_graph.py            ← DAG + TaskNode
    task_scheduler.py
    event_bus.py
    model_router.py
    intelligence_feed.py     ← OrchestratorIntelligenceFeed

  codebase/
```

```
    scanner.py                ← structural scan
    semantic_reader.py        ← Tree-sitter + LLM summarisation
    state_assessor.py         ← completion + broken import detection
    context_builder.py        ← assembles brief for planner
    conventions.py            ← style + pattern extraction

  execution/
    executor.py
    sandbox.py
    safety.py

  memory/
    manager.py
    vector_store.py
    session_logger.py
    fix_patterns.py

  prompt_engine/
    middleware.py
    rag_retriever.py
    quality_scorer.py
    templates/
      planner.md
      architect.md
      coder.md
      debugger.md
      reviewer.md
      memory_optimizer.md

  protocol/
    schemas.py                ← TaskResult, PatchRequest, ErrorReport, Agent

  rl/
    bandit.py                 ← Tier 1: UCB contextual bandit
    trajectory_logger.py      ← Tier 2: (state, action, reward) logging
    policy.py                 ← Tier 2: RoutingPolicy MLP
    reward.py                 ← composite reward function
    trainer.py                ← Behavior Cloning + CQL training loop
    encoder.py                ← task state → fixed-size vector

  tools/
    filesystem.py
    terminal.py
    git_tools.py

  scripts/
```

```
    sage-context.sh
    post-commit.sh
    memory_optimizer.sh

  cli/
    main.py                  ← sage run | sage status | sage memory | sage

  benchmarks/
    tasks/
      rest_api.yaml
      crud_app.yaml
      bug_fix.yaml
      continuation.yaml
      full_stack.yaml
    runner.py
    metrics.py

  config/
    models.yaml
    pipeline.yaml

  docs/
    architecture.md
    getting_started.md
    contributing.md
    research_notes.md
```

---

26. Development Roadmap

Phase 1 — Core Pipeline (Week 1)

Goal: `sage run "build FastAPI hello world"` produces working code

- [ ] LangGraph workflow: detect_mode → load_memory → planner → scheduler → coder → save_memory
- [ ] `sage-context.sh` + `system_state.json`
- [ ] Basic sequential Task DAG (no parallelism yet)
- [ ] obra/superpowers skill integration
- [ ] Git hook + `.sage-memory.md`
- [ ] CLI: `sage run "prompt"`

- [ ] session manager

Phase 2 — Execution + Debug Loop (Week 2)

Goal: Self-healing debug loop on simple projects

- [ ] Tool Execution Engine with safety limits
- [ ] Executor + pytest integration
- [ ] Debug Agent (Codestral)
- [ ] Fix Pattern Store
- [ ] user rules
- [ ] Circuit breaker (max 5 retries)
- [ ] Session journals + `MEMORY_CHECKPOINT`
- [ ] Tier 1 RL: UCB bandit on prompt strategy selection

Phase 3 — Intelligence Layers (Week 3)

Goal: Multi-task, supervised, context-aware

- [ ] Full DAG with dependency resolution
- [ ] Task Scheduler (MAX_PARALLEL=3)
- [ ] Async event bus
- [ ] Model Router (full routing table)
- [ ] Orchestrator Intelligence Feed + `AgentInsight` protocol
- [ ] Human-in-the-loop checkpoints (all 5 types)
- [ ] Codebase Intelligence Layer (scanner + state assessor)
- [ ] `.sage/` project directory

Phase 4 — Full Intelligence (Week 4)

Goal: Open-source ready, demo video recorded

- [ ] Prompt Intelligence Middleware (RAG over docs)
- [ ] Qdrant vector store
- [ ] Semantic code reader (Tree-sitter)
- [ ] Prompt quality delta measurement
- [ ] Memory Optimizer Agent + weekly cron
- [ ] Trajectory logging instrumented (for Tier 2 RL)

- [ ] Full benchmark suite: 6 task types, 8 metrics
- [ ] CLI: `sage bench`
- [ ] README, architecture diagram, contributing guide

Phase 5 — Offline RL (Post-launch, ~2 months)

Goal: Learned routing policy outperforms static table

- [ ] 500+ diverse trajectories collected
- [ ] Reward function tuned
- [ ] Behavior Cloning baseline trained
- [ ] CQL policy trained + integrated with fallback
- [ ] Benchmark: policy vs static table comparison

Phase 6 — Simulator RL (6+ months)

Goal: Benchmark-guided online RL

- [ ] 1000+ benchmark tasks with known solutions
- [ ] Parallel Docker sandbox infrastructure
- [ ] PPO fine-tuning pipeline

---

27. Minimum Viable System

```
detect_mode
      ↓
[existing repo] → scanner + state_assessor → context brief
[greenfield]    → skip
      ↓
sage-context.sh
      ↓
Planner (Llama 3) → Task DAG
      ↓
Task Scheduler (sequential, no parallelism)
      ↓
Coder (DeepSeek-Coder) → PatchRequest
  └─ emits AgentInsight (logged, not yet evaluated)
      ↓
```

```
Executor (subprocess, no Docker)
      ↓
Fix Pattern Store check
      ↓ (no match)
Debug Agent (Codestral) → patch → retry (max 5)
      ↓
Circuit Breaker → skip + log
      ↓
save_memory (system_state + session journal + git hook)
```

This MVP already outperforms most open-source agent systems:
existing repo support, circuit breaker, persistent memory,
structured protocol, fix pattern learning, safety limits.

---

28. Self-Improvement Loops

Loop 1 — Prompt Learning

```
Weak prompt
      ↓
Prompt Intelligence Middleware retrieves best known strategies
      ↓
Structured, model-specific, context-aware prompt
      ↓
Better output → quality score logged
      ↓
Successful prompts rank higher in future retrieval
```

Measured by: **prompt quality delta**

Loop 2 — Debug Learning

```
Error encountered
      ↓
Debug Agent fixes (first occurrence) → pattern stored
      ↓
Future identical error: skip debug loop, apply known fix directly
      ↓
Pattern success rate tracked via exponential moving average
```

```
      α = 0.2: new_rate = 0.8 × old_rate + 0.2 × latest_outcome
      Patterns below 0.4 → flagged for review
```

Measured by: **fix pattern hit rate**

The Compounding Mechanism

Fix patterns (Loop 2) are injected into the coder's prompt as `KNOWN PATTERNS`. The UCB bandit (Loop 1) learns that prompts with relevant fix patterns produce higher reward scores. Loop 2 improves Loop 1's inputs automatically.

```
More fix patterns → better coder prompts → fewer errors → faster runs
                                                              ↓
                                                   new patterns learned faster
```

Neither loop is independent. They compound.

Loop 3 — Orchestrator Learning (from Intel Feed)

```
AgentInsight emitted (medium severity)
      ↓
Orchestrator injects ORCHESTRATOR_NOTE into downstream prompt
      ↓
Downstream agent avoids the flagged issue
      ↓
Fewer task failures → fewer debug cycles
      ↓
Insight pattern logged: "when coder flags X, inject Y into reviewer"
```

Over time, the orchestrator builds a library of effective intervention patterns. This is a third, slower improvement loop that operates at the supervision layer.

---

29. Reinforcement Learning Strategy

SAGE implements RL in three tiers. Live online RL is deliberately excluded.

Why Not Live RL

Online RL requires exploration — the policy tries suboptimal actions to discover better ones. In SAGE this means deliberately routing tasks to wrong models and letting them fail. Combined with noisy rewards (failures from timeouts, JSON errors, Docker lag — not policy quality) and long action sequences making credit assignment nearly impossible, live RL degrades reliability for weeks before improving anything.

The only viable live RL setup requires 1000+ parallel isolated benchmark runs with a GPU training pipeline — a research lab setup, not a solo build. That is Phase 6.

Tier 1 — Contextual Bandit (Phase 2, immediate)

Applied to: prompt strategy selection + model routing
Algorithm: UCB (Upper Confidence Bound)

```
# score = quality_mean + sqrt(2 * log(total_tries) / tries_for_this_strateg
# High score = proven performer OR underexplored → worth trying
```

State	task_type + agent_role + complexity_score
Action	which prompt template + model
Reward	build_success + test_pass_rate − (0.1 × debug_cycles)

No training infrastructure. Updates a value table in memory.
Works from session 1. Zero regression risk.

Tier 2 — Offline RL on Orchestrator Decisions (Phase 5, ~2 months post-launch)

Applied to: full routing policy (model + template + agent assignment)
Algorithm: Behavior Cloning first → CQL (Conservative Q-Learning)

Trajectory Logging (instrument from Phase 4)

```
{
  "state": {
```

```
    "task_type": "create_api_endpoint",
    "complexity_score": 0.6,
    "language": "python",
    "repo_mode": "existing_repo",
    "project_context_embedding": [...]
  },
  "action": {
    "model_chosen": "deepseek-coder:6.7b",
    "prompt_template": "coder_v2",
    "agent_assigned": "coder"
  },
  "reward": 0.85,
  "next_state": { ... },
  "terminal": false
}
```

Composite reward:

```
build_success (1.0) + test_pass_rate (0–1)
- (0.1 × debug_cycles) - (0.5 × circuit_breaker_fire)
```

Policy Network (~50k parameters)

```
class RoutingPolicy(nn.Module):
    def __init__(self, state_dim=256, action_dim=12):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128), nn.ReLU(),
            nn.Linear(128, 64),        nn.ReLU(),
            nn.Linear(64, action_dim), nn.Softmax(dim=-1)
        )
```

Integration (static table always the fallback)

```
def route(self, task: TaskNode) -> str:
    state_vector = self.encoder.encode(task)
    action_probs = self.policy(state_vector)
    if action_probs.max() < self.confidence_threshold:
        return self.static_routing_table[task.agent_type]
    return self.action_space[action_probs.argmax()]
```

Timeline

Step	Effort	When
Instrument trajectory logging	2–3 days	Phase 4
Collect 500+ diverse trajectories	4–6 weeks	Post-launch
Tune reward function	3–4 days	Phase 5
Train Behavior Cloning baseline	1–2 days	Phase 5
Train CQL policy	1–2 days	Phase 5
Integrate + test fallback	2–3 days	Phase 5

Ship Behavior Cloning first — it works well and gives a baseline to beat.

Tier 3 — Simulator-Based Online RL (Phase 6, 6+ months)

Applied to: coding agent fine-tuning
Algorithm: PPO against benchmark simulator

Requires 1000+ benchmark tasks with known solutions,
100+ parallel Docker sandboxes, GPU fine-tuning infrastructure.
This is how Devin was trained.

RL × Intel Feed Connection

Circuit breaker activations (max retry events) are high-quality negative training examples for the Tier 2 policy. The Intel Feed's intervention log becomes a supervision signal: when intervention X prevented failure Y, that's a reward boost for the routing decision that triggered the intervention early.

---

30. Open Source Strategy

What Makes It Research-Grade

1. Reproducible benchmark suite across all four modes (greenfield, feature, bugfix, continuation)
2. Prompt quality delta measurement (middleware before/after)
3. Fix pattern hit rate learning curve (does it actually improve?)
4. Orchestrator intervention effectiveness (does the Intel Feed prevent failures?)
5. Ablation studies: with/without middleware, with/without fix patterns, with/without Intel Feed
6. Structured logging enabling full run reproduction

Release Plan

Release	Contents	Goal
v0.1	MVP pipeline (Phases 1-2)	GitHub stars
v0.2	Full system + codebase ingestion (Phases 3-4)	Developer adoption
v0.3	Benchmark results + research notes published	Citations
v0.4	Offline RL policy (Phase 5)	Research credibility

---

31. Long-Term Vision

SAGE becomes an **AI Software Engineering Lab** — a platform for researching how multi-agent systems can reliably write, extend, and repair software.

It works on the project you started six months ago and abandoned.
It works on the codebase you inherited with no documentation.
It works on the bug you've been avoiding for three weeks.
And it gets better at all of it every time it runs.

Three possible futures for SAGE:

- Research platform: published benchmarks, arxiv paper, community contributions
- Open-source ecosystem: community-built agents, model adapters, prompt libraries
- Startup foundation: the infrastructure layer others build their tools on

---

32. Quick Start

```
# Clone and setup
git clone https://github.com/yourusername/sage
cd sage
pip install -r requirements.txt
ollama pull deepseek-coder:6.7b llama3:8b codestral

# Scaffold memory
mkdir -p memory/{sessions,projects,fixes,weekly}
echo '{}' > memory/system_state.json

# Install git hook
cp scripts/post-commit.sh .git/hooks/post-commit
chmod +x .git/hooks/post-commit

# Greenfield
sage run "Build a FastAPI backend with JWT auth and PostgreSQL"

# Existing repo
sage run "add rate limiting to the login endpoint" --repo ./my-project

# Unfinished repo
sage run "finish what was started" --repo ./abandoned-project

# Research mode (all checkpoints)
sage run "build a REST API" --research

# Check what SAGE knows about your project
sage memory

# Run benchmarks
sage bench
```

SAGE — Self-improving Autonomous Generation Engine

prompt → production

END OF SPECIFICATION v1.0 FINAL

Checkpoint 1 complete. All architectural decisions locked.

Next: Phase 1 implementation — LangGraph skeleton + protocol schemas.